

Team Details:

Team Leader: SANDHYA R

Team Members: REKHA P, SARITHA R, SAVITHRI K, SINDHUJA S

Answer Bot- AI-powered FAQ and customer support automation

Introduction

The AI FAQ Assistant is a robust, secure, and intelligent REST API platform designed to streamline FAQ creation, user authentication, and AI-powered content management. Built with Node.js and Express.js, the application utilizes MongoDB as its primary data store along with Mongoose ODM for elegant data modeling and schema validation.

Security is central to the architecture, implementing JWT (JSON Web Tokens) for stateless user sessions and route guarding alongside bcrypt for strong password encryption. The backend handles intensive operations like automated FAQ generation, semantic search queries, and dynamic context parsing. It also features robust centralized error handling and response status sanitization to prevent unhandled database leakage and ensure operational durability.

Project Overview

The **AI FAQ Assistant API** is an intelligent application designed to answer frequently asked questions automatically. Instead of manually searching through documents or asking a human support person, users can send a question to the API and receive a relevant answer.

The system provides a backend API that accepts user questions, processes them using AI/NLP techniques, searches the available FAQ information, and returns the most appropriate answer.

Main Objective

The main objective of this project is to develop an **AI-powered FAQ system** that can:

- Accept questions from users.
- Understand the user's question.
- Identify the relevant FAQ.
- Generate or retrieve an appropriate answer.
- Return the answer through an API.
- Reduce repetitive manual support work.

1. Brainstorming & Ideation Phase

1.1 Problem Identification

In many organizations, colleges, hospitals, companies, and online services, users repeatedly ask similar questions.

For example:

- What are the working hours?
- How can I register?
- How can I reset my password?
- What documents are required?
- How can I contact support?
- What is the admission procedure?

Answering the same questions repeatedly requires human effort and time.

Therefore, an automated FAQ assistant can be developed to provide answers quickly.

1.2 Proposed Solution

The proposed solution is an **AI FAQ Assistant API**.

The system allows an application or website to send a question to the API.

1.3 Idea

The main idea is to create a reusable API that can be connected to:

- Websites
- Mobile applications
- College portals
- Customer-support systems
- Chat applications
- Business applications

1.4 Innovation

The system improves traditional FAQ systems by allowing users to ask questions in natural language instead of searching manually through a list of questions.

Example

Traditional FAQ:

User searches: "Password Reset"

AI FAQ Assistant:

User asks: "I forgot my password. How can I change it?"

The system identifies the meaning and provides the relevant answer.

1.5 Expected Benefits

- Saves time.
- Reduces repetitive questions.
- Provides 24/7 assistance.
- Gives consistent answers.
- Easy to integrate with other applications.
- Reduces workload for support staff.
- Improves user experience.

2. Requirement Analysis Phase

2.1 Functional Requirements

The system should provide the following functions:

1. Question Input

The user/application should be able to send a question.

2. Question Processing

The API should process the received question.

3. FAQ Search

The system should search the available FAQ/knowledge base.

4. Answer Generation

The system should identify or generate an appropriate answer.

5. API Response

The answer should be returned in a structured format such as JSON.

6. Error Handling

The system should provide an appropriate message when:

- Question is empty.
- Question cannot be understood.
- No relevant FAQ is found.
- Server/API error occurs.

2.2 Non-Functional Requirements

Performance

The API should provide responses within a reasonable time.

Security

The API should protect sensitive information and validate incoming requests.

Reliability

The system should handle invalid requests without crashing.

Scalability

The application should support increasing numbers of users and questions.

Maintainability

The code should be organized so that future developers can easily modify it.

Usability

The API should be simple to use and integrate.

2.3 Software Requirements

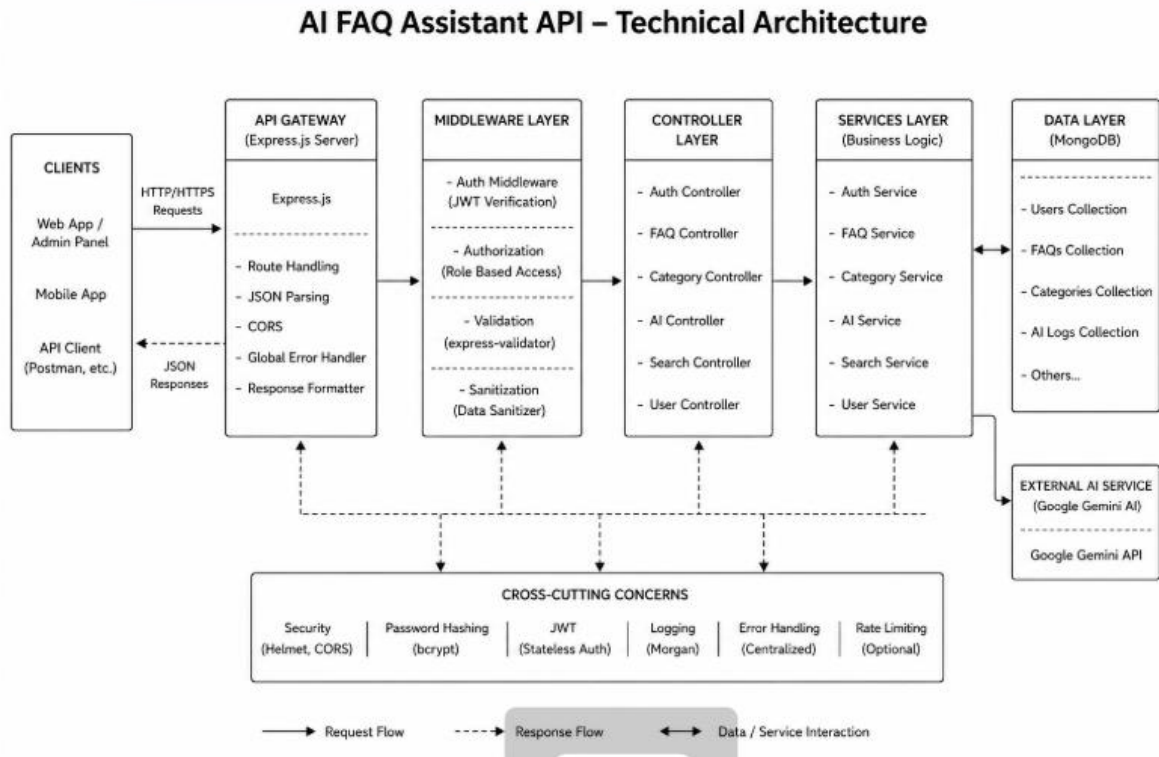
- Operating System: Windows 10/11, macOS, or Linux (supports cross-platform operations).
- Node.js (v18 or above): Runtime ecosystem running server logic and routing infrastructure.
- npm (v9 or above): Node package manager to control operational dependencies.
- Express.js (v4/v5 Ecosystem): Lightweight routing web framework to construct backend RESTful entry points.
- MongoDB / Mongoose ODM: Document NoSQL engine storing application users and curated data entities.
- Postman / ThunderClient: API verification toolkit to assert schema validation responses across administrative route shields.
- Code Editor: Visual Studio Code or similar IDE environment.

2.3 Hardware Requirements

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB processing layers).
- Storage: 1 GB of available disk workspace.

3. Project Design Phase

3.1 Project Architecture



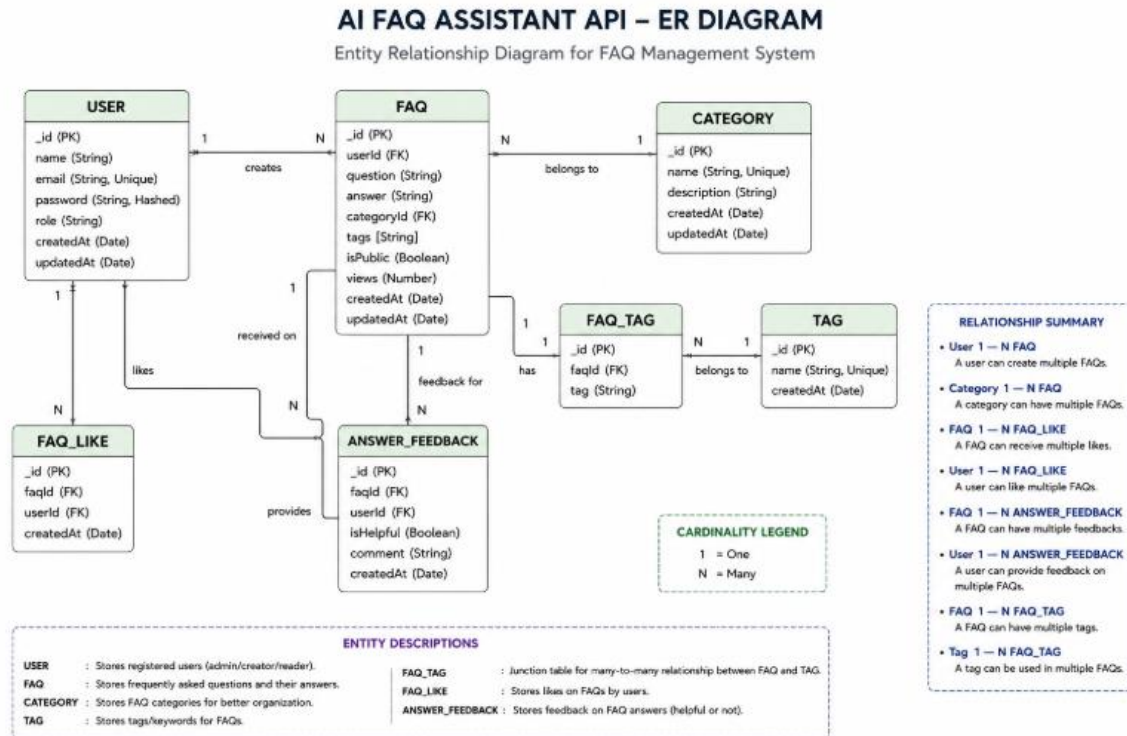
The AI FAQ Assistant application exposes structured endpoints handled by a layered server architecture model. Incoming requests undergo filter processing before interacting with data services.

3.2 project Modules

- **Interface:** API tools like Postman query distinct route paths to extract JSON entities.
- **Express Server Gateway:** Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses incoming JSON payloads, and handles CORS headers.
- **Authentication Middleware:** Extracts tokens from headers, validates user context signatures, and evaluates authorization thresholds for guarded resource paths.
- **Granular Services (Logic Modules):** Enforces isolated domain rules for user session life cycles and operational query modifications.
- **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries.

- AI Flow (Service Integration): Connected to external AI models via Google Gemini SDK integrations, letting the controller layer offload complex question-answering tasks and automated topic generation workflows.

3.3 Entity-Relationship (ER) Diagram Description



The backend database consists of multiple MongoDB collections connected through logical references to maintain user authentication, FAQ management, AI-generated content, and category organization.

1. User Entity

- `_id` (Primary Key - ObjectId)
- `name` (String, Required)
- `email` (String, Required, Unique)
- `password` (String, Hashed, Required)
- `role` (String, Default: "user")
- `createdAt`
- `updatedAt`

2. FAQ Entity

- `_id` (Primary Key - ObjectId)
- `question` (String, Required)
- `answer` (String, Required)
- `category` (String, Required)
- `createdBy` (Foreign Key referencing User Collection)
- `createdAt`
- `updatedAt`

3. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI before persistence.)

- `topic` (String)
- `generatedQuestion` (String)
- `generatedAnswer` (String)
- `generatedCategory` (String)
- `generatedAt`

4. Category Entity (Logical)

- `categoryName` (String)
- `description` (String)
- `totalFAQs` (Number)

Key Features

- **JWT Authentication & Authorization:** Secure token-based authentication protects private API endpoints.
- **AI FAQ Generation:** Google Gemini AI automatically generates questions, answers, and topic suggestions.
- **Semantic FAQ Search:** Intelligent keyword-based search enables faster information retrieval.
- **Complete CRUD Operations:** Create, Read, Update, and Delete FAQ records.
- **Category Management:** Organizes FAQs into structured categories for easier navigation.
- **Password Encryption:** User passwords are securely hashed using bcrypt before storage.

- **Centralized Error Handling:** Standardized API error responses improve reliability and debugging.
- **Schema Validation:** Mongoose validation ensures data integrity before database persistence.
- **RESTful API Architecture:** Modular Express.js routes provide scalable backend services.
- **MongoDB Integration:** Efficient NoSQL document storage using Mongoose ODM.

Roles and Responsibilities

Admin

- Manage all registered users.
- View, update, and delete any FAQ.
- Monitor AI-generated content.
- Manage FAQ categories.
- Access complete system functionality.

Content Creator

- Create new FAQs.
- Edit personal FAQ records.
- Delete owned FAQs.
- Generate FAQs using AI.
- Organize FAQs under categories.

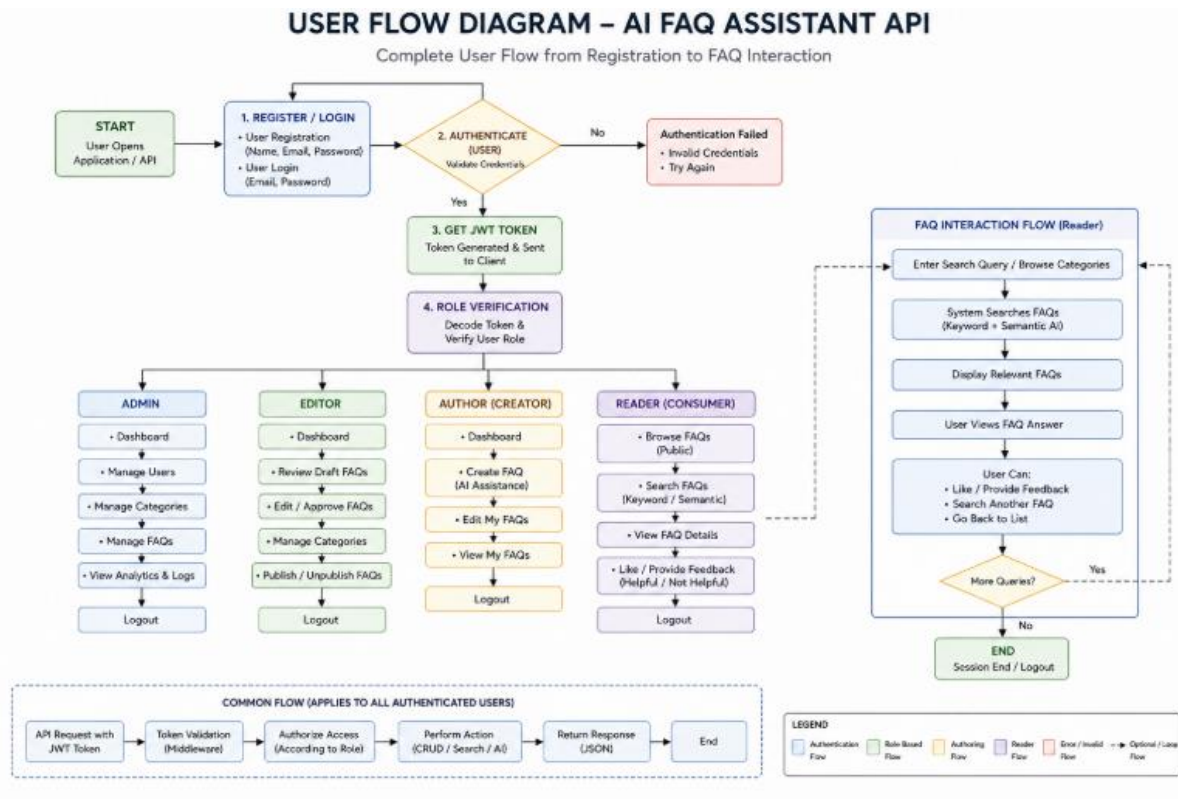
Authenticated User

- View public FAQs.
- Search FAQs using keywords.
- Generate AI-assisted answers.
- Access personal profile information.

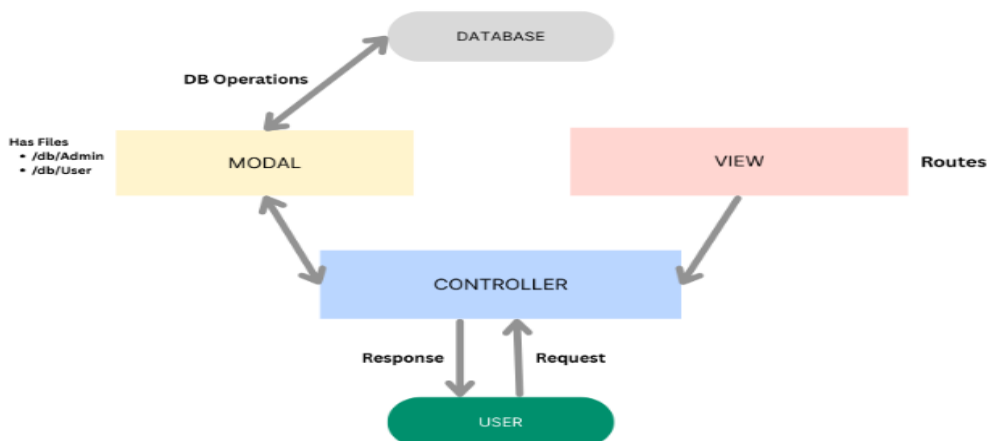
Public User

- View published FAQs.
- Search FAQs.
- Cannot create, edit, or delete FAQ records.
- No access to protected API endpoints.

User Flow



MVC Architecture Pattern : The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:

Model Layer (Data Blueprint)

Defines structural blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets field rules and defaults before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from execution routes, checks structural request constraints, delegates processing workflows directly to models or services, and packages resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.

4. Project Planning Phase

4.1 Project Activities

The project can be divided into the following activities:

1. Problem identification
2. Requirement collection
3. Technology selection
4. System design
5. Database design
6. API development
7. AI integration
8. Testing
9. Documentation
10. Demonstration

4.2 Team Responsibilities

Possible responsibilities:

Team Member 1

- Requirement analysis
- Documentation

Team Member 2

- Backend/API development

Team Member 3

- Database development

Team Member 4

- AI integration
- Testing

Responsibilities can be changed according to the actual team

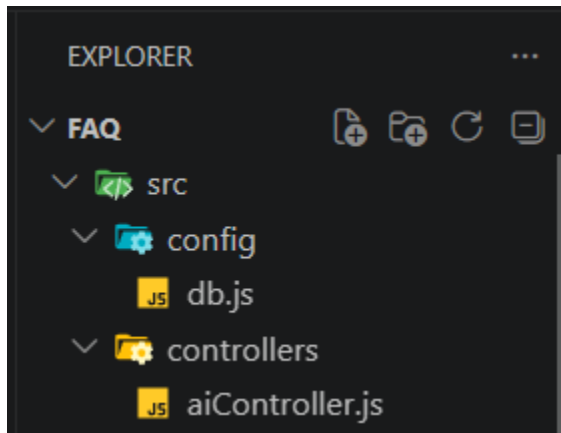
4.3 Risk Management

Risk	Solution
API failure	Implement error handling
Database failure	Backup database
Incorrect answer	Improve FAQ/AI data
Invalid input	Input validation
Slow response	Optimize API/database
Security issue	Authentication and validation

5. Project Development Phase

5.1 Creating Project Folder

1. Create a new folder with your <project name>.
2. Inside that folder, create a new subfolder named src.
3. Now open that folder in VS Code.



Server Setup (npm init)

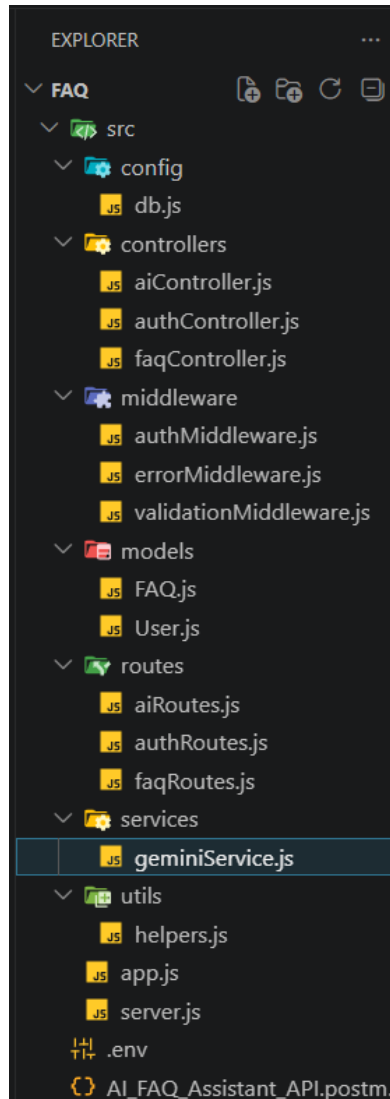
Open your root folder terminal in VS Code and run:

Bash

```
JavaScript
npm init -y
npm install express mongoose dotenv cors jsonwebtoken
bcrypt @google/genai
```

File and Folder Structure Matrix

Plaintext



5.2 Backend Development

Core System Script Implementations

1. Gateway Server Node (src/server.js)

This main entrance mechanism aggregates environment variables, triggers database listeners, and binds network listeners to active environment ports:

```

c> serverjs > ...
1 const app = require('./app');
2 const connectDB = async () => {
3   try {
4     const dbConnect = require('./config/db');
5     await dbConnect();
6   } catch (error) {
7     console.error('Failed to connect to the database', error);
8   }
9 };
10
11 const startServer = async () => {
12   // Connect to database
13   await connectDB();
14
15   const PORT = process.env.PORT || 5000;
16
17   const server = app.listen(PORT, () => {
18     console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
19   });
20
21   // Handle unhandled promise rejections
22   process.on('unhandledRejection', (err, promise) => {
23     console.error('Unhandled Rejection: ${err.message}');
24     // Close server & exit process
25     server.close(() => process.exit(1));
26   });
27 };
28
29 startServer();
30

```

2. Gateway Express Setup (src/app.js)

```

src> appjs > ...
1 const express = require('express');
2 const cors = require('cors');
3 const dotenv = require('dotenv');
4 const { errorHandler } = require('./middleware/errorMiddleware');
5
6 // Load environment variables
7 dotenv.config();
8
9 const app = express();
10
11 // Enable CORS
12 app.use(cors());
13
14 // Body parser middlewares
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: false }));
17
18 // Basic status check route
19 app.get('/', (req, res) => {
20   res.json({
21     success: true,
22     message: 'AI FAQ Assistant API is running.',
23     version: '1.0.0',
24   });
25 });
26
27 // Register API Routes
28 app.use('/api/auth', require('./routes/authRoutes'));
29 app.use('/api/faqs', require('./routes/faqRoutes'));
30 app.use('/api/ai', require('./routes/aiRoutes'));
31
32 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
33 app.use('/socket.io', (req, res) => {
34   res.status(404).json({ error: 'Socket.IO not enabled' });
35 });
36

```

3. Security Access Middleware (src/middleware/authMiddleware.js)

Intercepts incoming target queries, extracts transmission header authorization records, and validates token identity context claims:

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:

```

src > models > User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcrypt');
3
4  const userSchema = new mongoose.Schema(
5  {
6    name: {
7      type: String,
8      required: [true, 'Name is required'],
9      trim: true,
10   },
11   email: {
12     type: String,
13     required: [true, 'Email is required'],
14     unique: true,
15     lowercase: true,
16     trim: true,
17     match: [/^([a-zA-Z0-9_\.\-])+\@([a-zA-Z0-9_\.\-])+\./, 'Please provide a valid email address'],
18   },
19   password: {
20     type: String,
21     required: [true, 'Password is required'],
22     minlength: [6, 'Password must be at least 6 characters long'],
23   },
24 },
25 {
26   timestamps: true,
27 }
28 );
29
30 // Pre-save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });

```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:

```

src > models > FAQ.js > ...
1  const mongoose = require('mongoose');
2
3  const faqSchema = new mongoose.Schema(
4  {
5    question: {
6      type: String,
7      required: [true, 'Question is required'],
8      trim: true,
9    },
10   answer: {
11     type: String,
12     required: [true, 'Answer is required'],
13     trim: true,
14   },
15   category: {
16     type: String,
17     required: [true, 'Category is required'],
18     enum: {
19       values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20       message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking,
21     },
22     default: 'General',
23   },
24   createdBy: {
25     type: mongoose.Schema.Types.ObjectId,
26     ref: 'User',
27     required: [true, 'Creator user reference is required'],
28   },
29 },
30 {
31   timestamps: true,
32 }
33 );
34
35 const FAQ = mongoose.model('FAQ', faqSchema);
36

```

API Router Interface Endpoints

1. Public Authentication Operations (src/routes/authRoutes.js)

```
.env  geminiService.js  authRoutes.js  AI_FAQ_Assistant_API.postman_collection.json  B...
src > routes > authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    registerUser,
5    loginUser,
6    getUserProfile,
7  } = require('../controllers/authController');
8  const { protect } = require('../middleware/authMiddleware');
9  const {
10    validateRegister,
11    validateLogin,
12  } = require('../middleware/validationMiddleware');
13
14  // Public routes
15  router.post('/register', validateRegister, registerUser);
16  router.post('/login', validateLogin, loginUser);
17
18  // Protected routes
19  router.get('/profile', protect, getUserProfile);
20
21  module.exports = router;
22
```

2. Protected FAQ Operations (src/routes/faqRoutes.js)

JavaScript

```
.env  geminiService.js  faqRoutes.js  AI_FAQ_Assistant_API.postman_collection.json
> routes > faqRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    createFAQ,
5    getAllFAQs,
6    getFAQById,
7    updateFAQ,
8    deleteFAQ,
9    searchFAQs,
10  } = require('../controllers/faqController');
11  const { protect } = require('../middleware/authMiddleware');
12  const { validateFAQ } = require('../middleware/validationMiddleware');
13
14  // Public routes - GET operations
15  router.get('/search', searchFAQs); // Note: defined before /:id to prevent routing issues
16  router.get('/', getAllFAQs);
17  router.get('/:id', getFAQById);
18
19  // Protected routes - Modification operations
20  router.post('/', protect, validateFAQ, createFAQ);
21  router.put('/:id', protect, updateFAQ);
22  router.delete('/:id', protect, deleteFAQ);
23
24  module.exports = router;
25
```

3. AI Automation Operations (src/routes/aiRoutes.js)

```
rc > routes > aiRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    generateAIAnswer,
5    generateAIFAQ,
6  } = require('../controllers/aiController');
7  const { protect } = require('../middleware/authMiddleware');
8  const {
9    validateAIAnswer,
10   validateAIFAQ,
11 } = require('../middleware/validationMiddleware');
12
13 // Protected routes to prevent API overuse/abuse
14 router.post('/answer', protect, validateAIAnswer, generateAIAnswer);
15 router.post('/generate-faq', protect, validateAIFAQ, generateAIFAQ);
16
17 module.exports = router;
18
```

6. Project Testing Phase

Running the API Application

- Install Postman or Thunder Client (VS Code extension).
- Start the backend development server environment parameters:

Bash

```
JavaScript  
npm run dev
```

API Testing Specification Demo

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

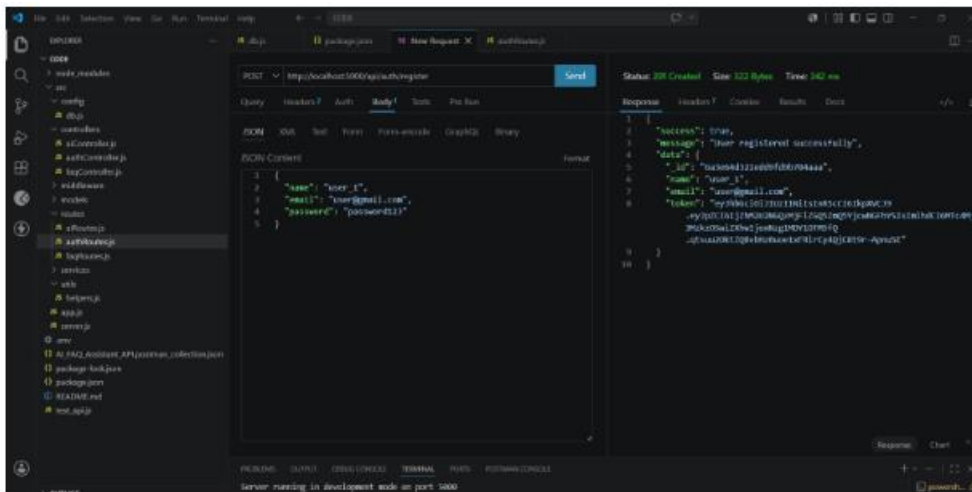
1. Identity Onboarding Path Registration

- Route: POST /api/auth/register
- Access: Public
- Payload Request Body JSON:

JSON

```
{  
  "name": "Priya Sharma",  
  "email": "priya@writeflow.com",  
  "password": "securepassword123"  
}
```

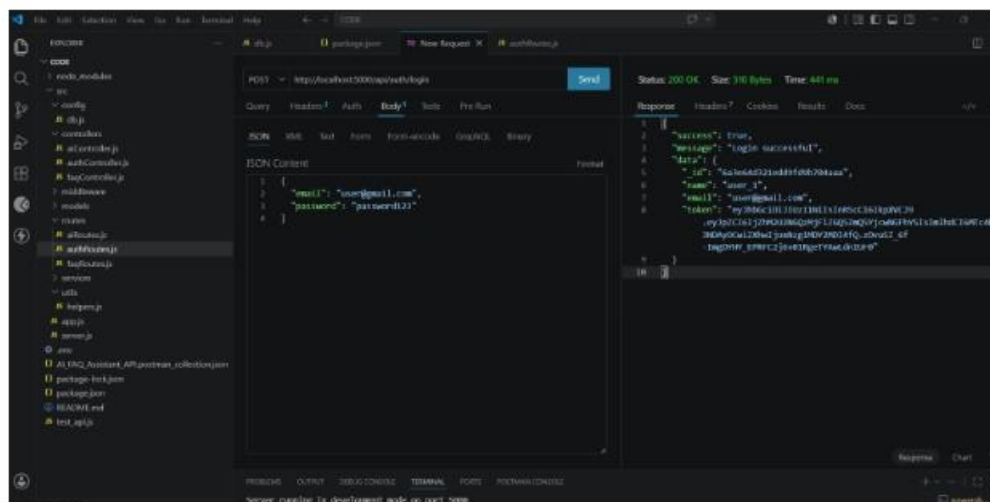
2. Profile Login Verification Quey



- Route: POST /api/auth/login
- Access: Public
- Payload Request Body JSON:

JSON

```
{
  "email": "priva@writeflow.com",
  "password": "securepassword123"
}
```

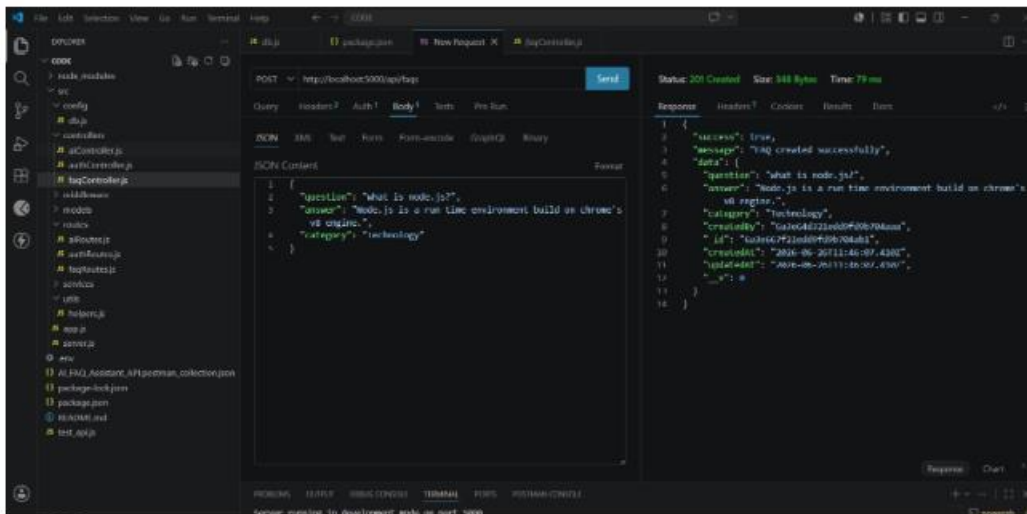


3. Resource Entity Creation (FAQ)

- Route: POST /api/faqs
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

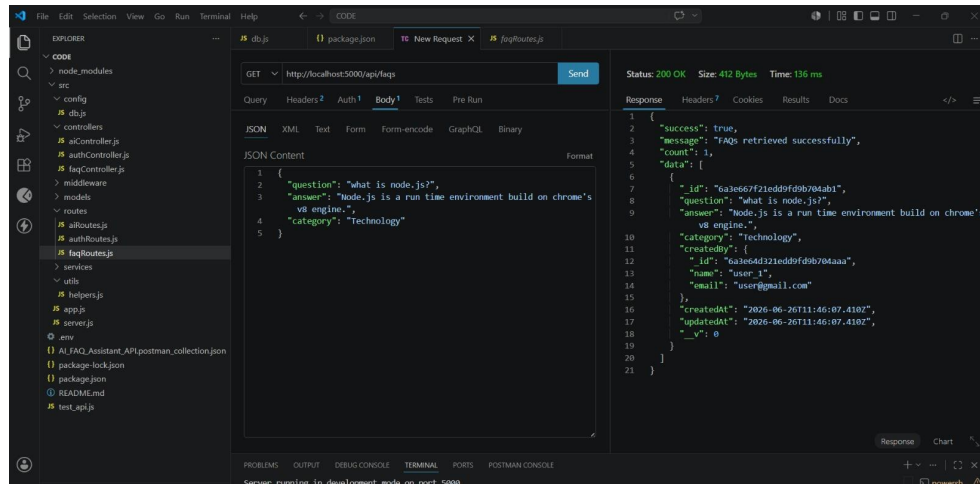
JSON

```
{  
  "question": "How do I configure custom category tags?",  
  "answer": "Navigate to settings panel, append keyword  
tags arrays, and save schema.",  
  "category": "Configuration"  
}
```



4. Keyword Resource Entity Fetching (Search)

- Route: GET /api/faqs/search?q=configure
- Access: Public



5. AI Intelligent Content Automation (Generate FAQ Pair)

- Route: POST /api/ai/generate-faq
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

JSON

JavaScript

```
{  
  "topic": "Mongoose schema indexing validation runtime  
workflow optimization"  
}
```

7. Project Documentation Phase

7.1 Project Abstract

AI FAQ Assistant API is an AI-based application designed to provide automated answers to frequently asked questions. The system receives user questions through an API, processes the questions using AI/NLP techniques, searches the available FAQ knowledge base, and returns a relevant response. The API architecture allows the system to be integrated with websites, mobile applications, and other software applications. The project reduces repetitive manual support activities and provides users with quick and consistent information.

7.2 Introduction

Frequently Asked Questions are common in many organizations and applications. Users often need answers to the same questions repeatedly. Traditional FAQ systems require users to manually search through a list of questions.

The AI FAQ Assistant API provides an intelligent alternative by allowing users to ask questions naturally. The backend processes the question and provides a suitable answer through an API.

7.3 Problem Statement

Users spend time searching for answers to frequently asked questions, while support teams spend time answering repetitive queries. A system is required to automate the process of answering common questions quickly and efficiently.

7.4 Proposed System

The proposed system is an AI-powered FAQ assistant that:

1. Receives a question.
2. Validates the request.
3. Processes the question.
4. Identifies the relevant information.
5. Retrieves or generates an answer.
6. Sends the answer back through the API

7.5 Objectives

- Develop an AI-powered FAQ API.
- Automate frequently asked questions.
- Reduce manual support.
- Provide fast responses.
- Improve accessibility of information.
- Create a reusable API.
- Provide structured JSON responses.

7.6 Advantages

- Easy to use.
- Fast response.
- Reduces human workload.
- Available at any time.
- Can be integrated with other applications.
- Centralized FAQ management.
- Scalable architecture.

7.7 Limitations

- The quality of answers depends on the FAQ/AI data.
- Very complex questions may require human support.
- AI services may require internet connectivity.
- API availability depends on the server.
- Incorrect or incomplete knowledge-base information can produce unsuitable answers.

7.8 Future Enhancements

Future versions can include:

- Voice-based questions.
- Multiple-language support.
- User authentication.
- Admin dashboard.
- Chat history.
- Analytics dashboard.
- Feedback system.

- More advanced AI models.
- Integration with websites and mobile applications.
- Automatic FAQ generation.
- Human-agent escalation.

7.9 Conclusion

The **AI FAQ Assistant API** provides an automated approach to answering frequently asked questions. By combining an API backend with AI/NLP processing and a structured FAQ knowledge base, the system can provide quick and consistent responses. The project can be extended to different domains such as education, healthcare, customer service, banking, and business applications.

8. Project Demonstration Phase

During the final demonstration, the project can be presented in the following order.

Step 1 – Introduce the Project

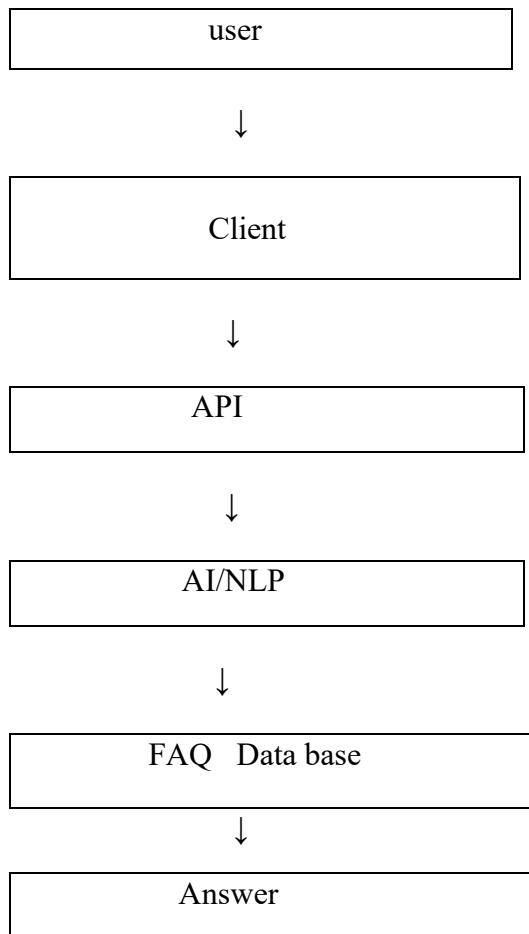
Project Title: AI FAQ Assistant API

Explain:

- What the project does.
- Why it is needed.
- Who can use it.

Step 2 – Explain Architecture

Show:



Step 3 – Start the Backend

Run the backend application.

Example:

```
Server started successfully  
Running on localhost
```

Step 4 – Open Postman

Create:

POST /api/faq/ask

Enter:

```
{  
  "question": "How can I reset my password?"  
}
```

Step 5 – Send Request

Click **Send**.

Expected result:

```
{  
  "success": true,  
  "answer": "You can reset your password using the Forgot Password  
option."  
}
```

Step 6 – Demonstrate Error Handling

Send:

```
{  
  "question": ""  
}
```

Show:

```
{  
  "success": false,  
  "message": "Question cannot be empty."  
}
```

Step 7 - Demonstrate Multiple Questions

Try:

What are the working hours?

How can I contact support?

How do I reset my password?

Show that the API provides the appropriate responses.

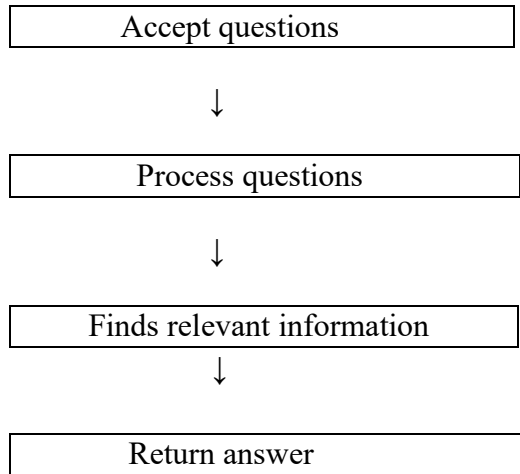
Step 8 – Explain GitHub

Show:

- Source code
- README
- Documentation
- Database files/scripts
- Testing files
- Screenshots

Step 9 – Final Result

Conclude the demonstration by showing that the system:



Final Project Flow

